

INSTITUTO TECNOLÓGICO SUDAMERICANO

Autor:

Jairo Arevalo

Asignatura:

Herramientas informáticas para el despliegue de diagramas

Actividad:

Realizar un backen en la cual se utilize recursos temporales

Carrera:

Desarrollo de Software

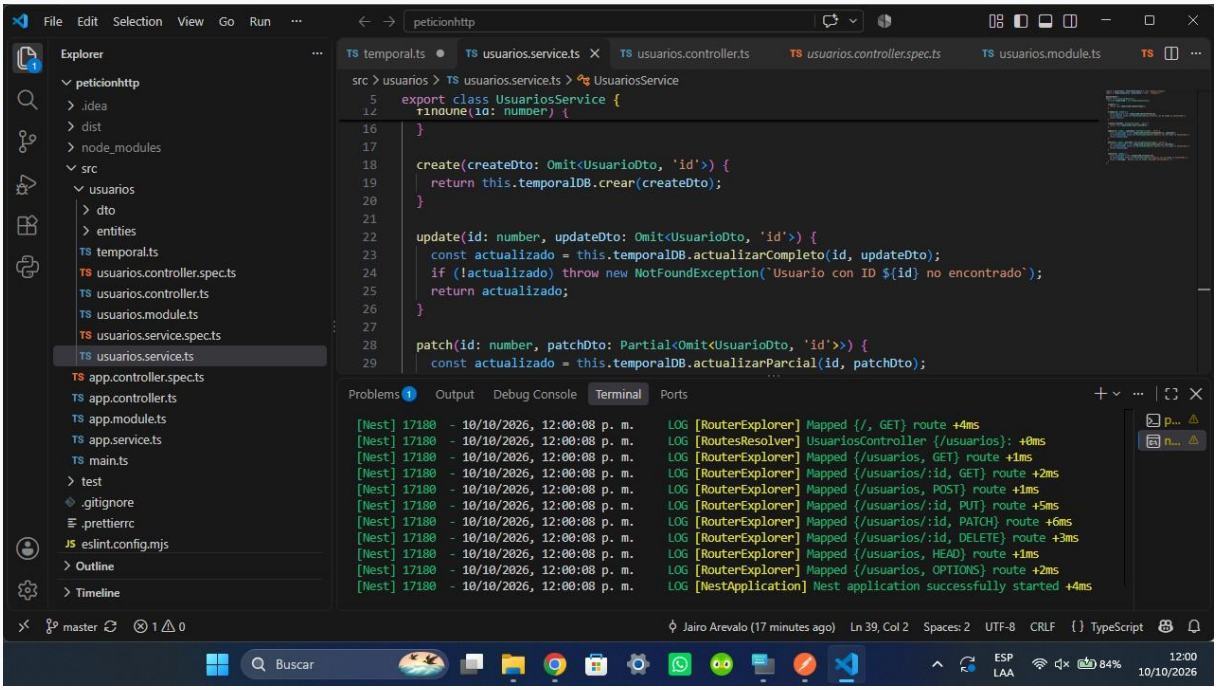
Clase:

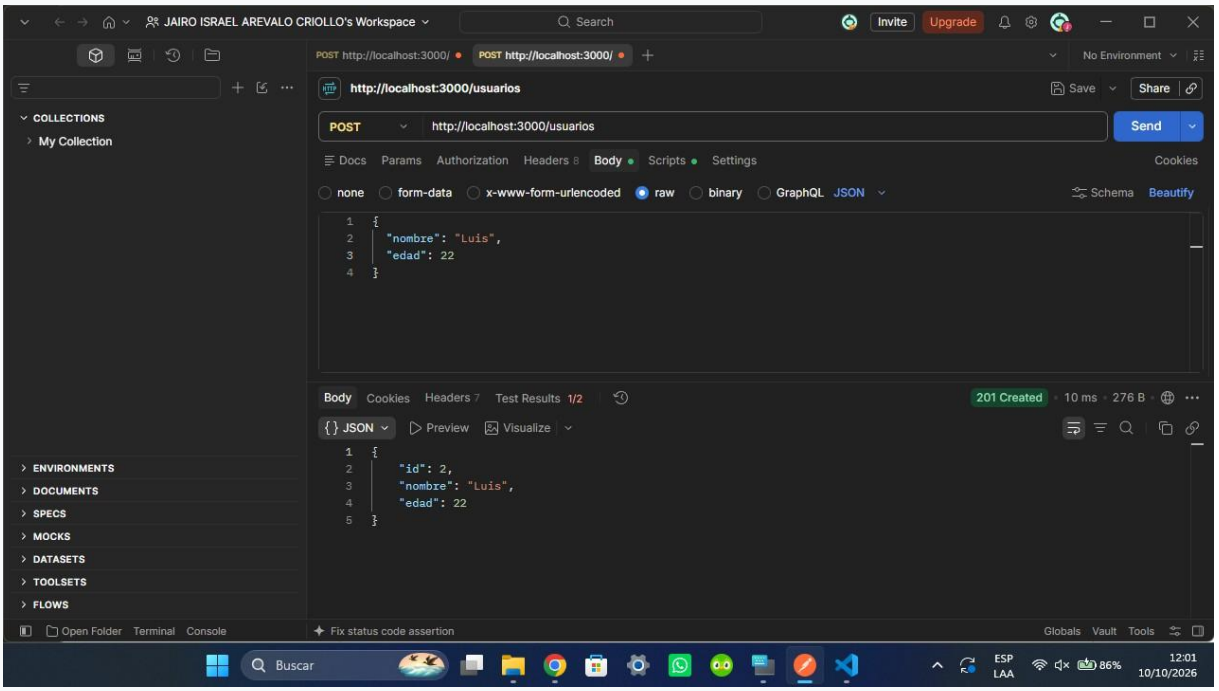
3er Ciclo

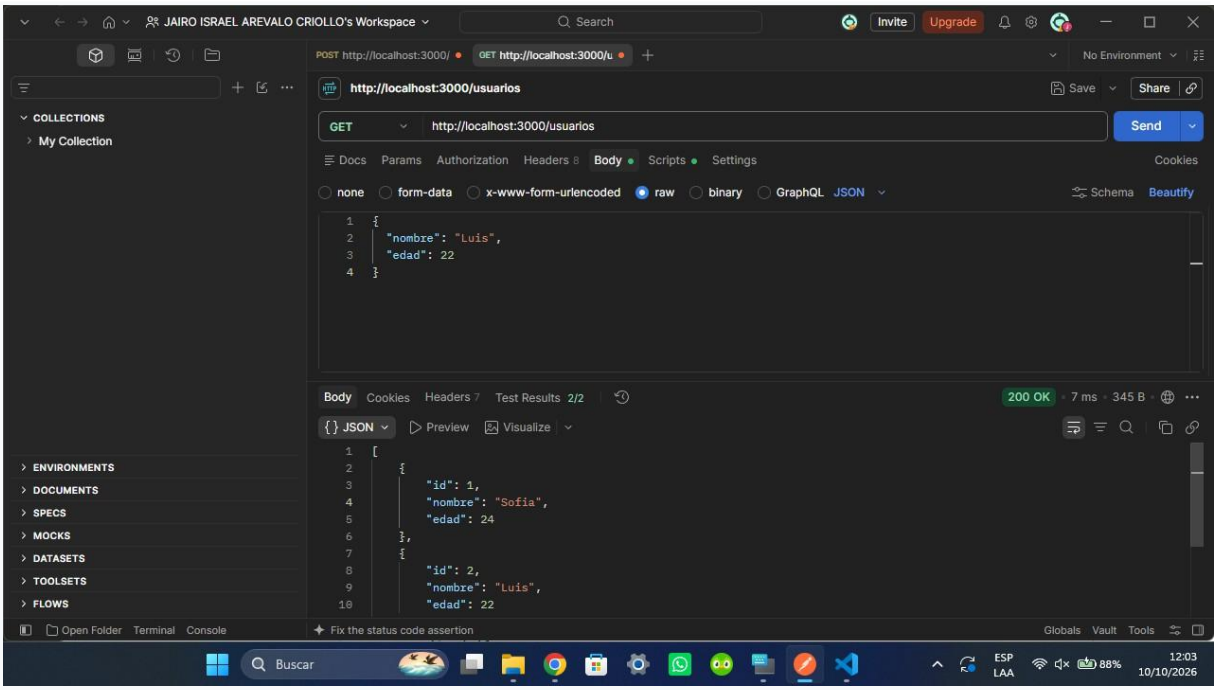
Fecha:

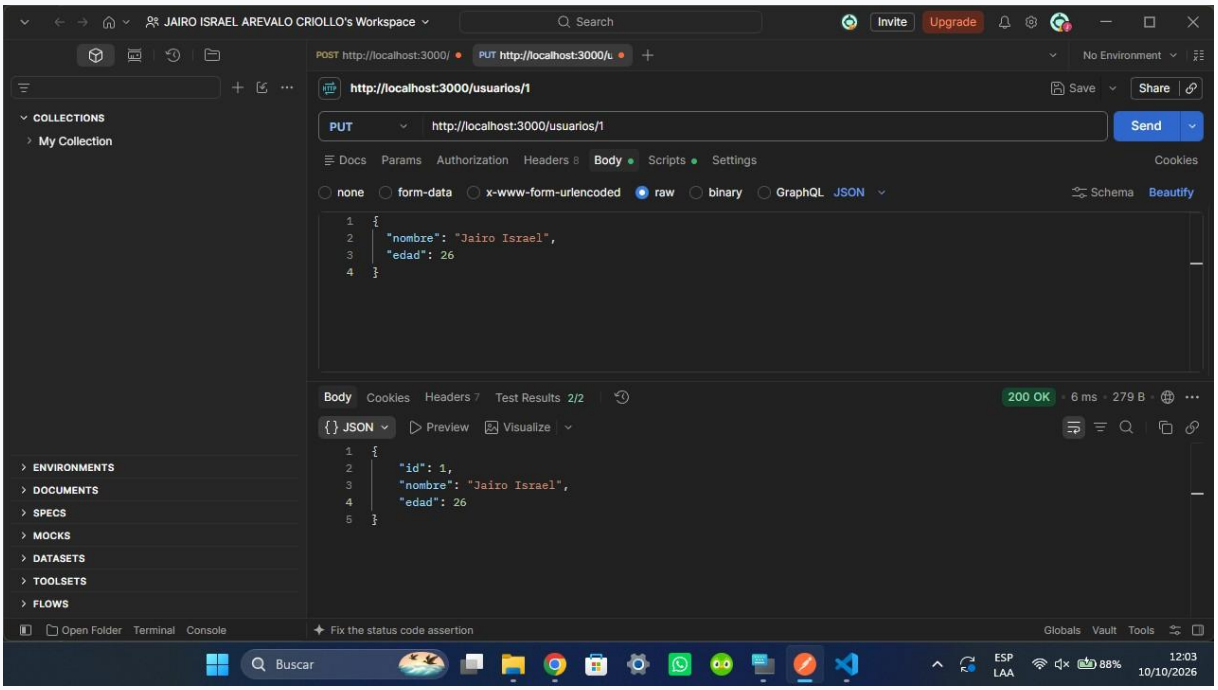
10 de octubre del 2026

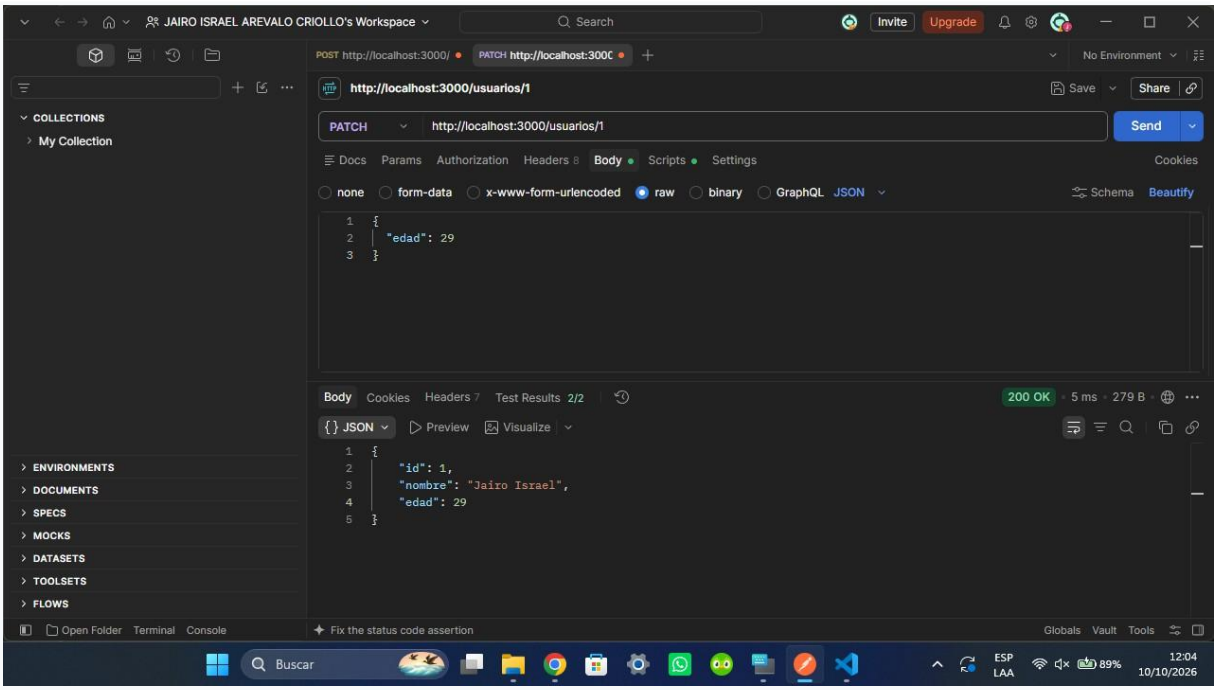
Enlace repositorio: https://github.com/jiarevalo-hub/peticionhttp

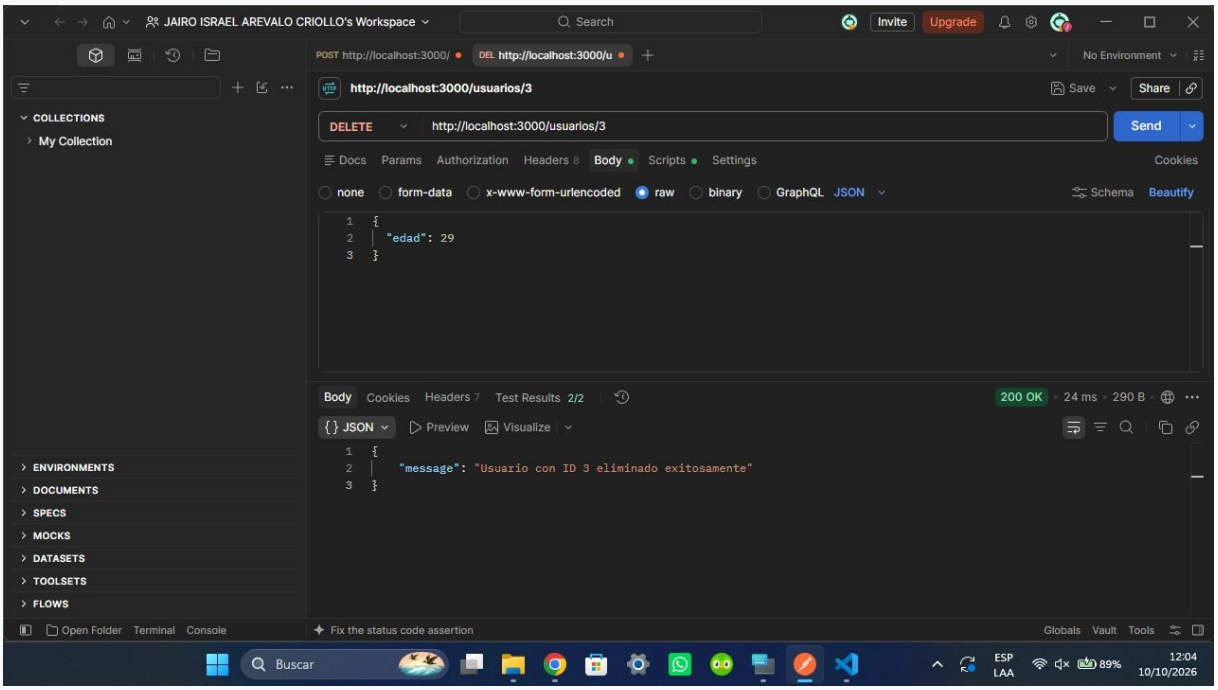


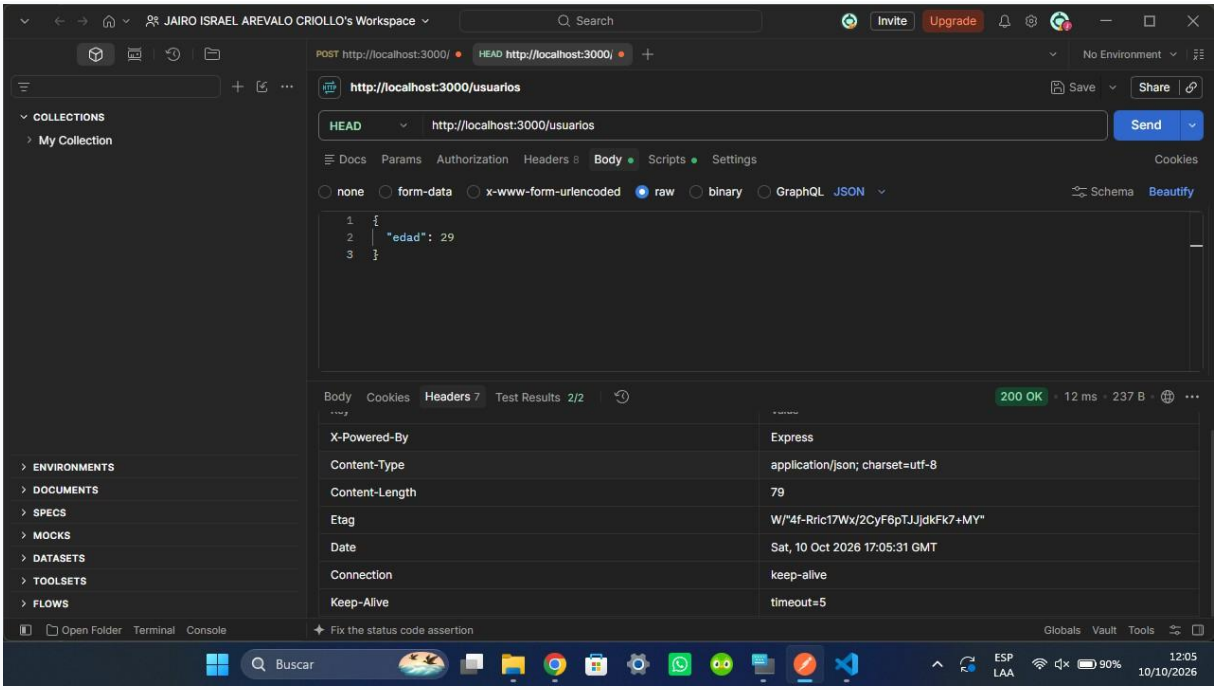


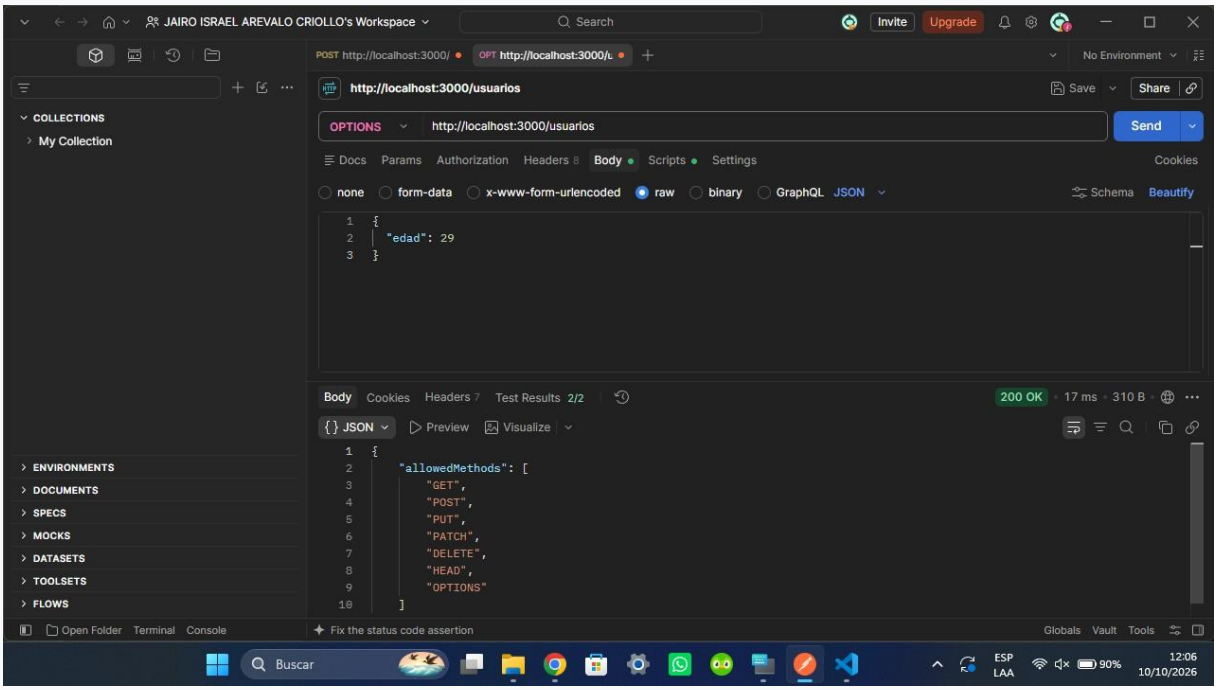

JAIRO ISRAEL AREVALO CRIOLLO's Workspace
Search
Invite
Upgrade
POST http://localhost:3000/
GET http://localhost:3000/u
No Environment
http://localhost:3000/usuarios
Save
Share
COLLECTIONS
My Collection
GET
http://localhost:3000/usuarios
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"nombre": "Luis",
"edad": 22
}
Body
Cookies
Headers 7
Test Results 2/2
200 OK
7 ms
345 B
{} JSON
Preview
Visualize
[
{
"id": 1,
"nombre": "Sofia",
"edad": 24
},
{
"id": 2,
"nombre": "Luis",
"edad": 22
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
Fix the status code assertion
Globals
Vault
Tools
Buscar
ESP
LAA
88%
12:03
10/10/2026


JAIRO ISRAEL AREVALO CRIOLLO's Workspace
Search
Invite
Upgrade
POST http://localhost:3000/
PUT http://localhost:3000/u
No Environment
http://localhost:3000/usuarios/1
Save
Share
COLLECTIONS
My Collection
PUT
http://localhost:3000/usuarios/1
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"nombre": "Jairo Israel",
"edad": 26
}
Body
Cookies
Headers 7
Test Results 2/2
200 OK
6 ms
279 B
{} JSON
Preview
Visualize
{
"id": 1,
"nombre": "Jairo Israel",
"edad": 26
}
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
Fix the status code assertion
Globals
Vault
Tools
Buscar
ESP
LAA
88%
12:03
10/10/2026


JAIRO ISRAEL AREVALO CRIOLLO's Workspace
Search
Invite
Upgrade
POST http://localhost:3000/
PATCH http://localhost:3000
No Environment
http://localhost:3000/usuarios/1
Save
Share
COLLECTIONS
My Collection
PATCH
http://localhost:3000/usuarios/1
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"edad": 29
}
Body Cookies Headers 7 Test Results 2/2
200 OK 5 ms 279 B
{ } JSON Preview Visualize
{
"id": 1,
"nombre": "Jairo Israel",
"edad": 29
}
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder Terminal Console
Fix the status code assertion
Globals Vault Tools
Buscar
ESP
LAA
89%
12:04
10/10/2026


JAIRO ISRAEL AREVALO CRIOLLO's Workspace
Search
Invite
Upgrade
POST http://localhost:3000/
DEL http://localhost:3000/u
No Environment
http://localhost:3000/usuarios/3
Save
Share
COLLECTIONS
My Collection
DELETE
http://localhost:3000/usuarios/3
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"edad": 29
}
Body Cookies Headers 7 Test Results 2/2
200 OK 24 ms 290 B
{ } JSON Preview Visualize
{
"message": "Usuario con ID 3 eliminado exitosamente"
}
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder Terminal Console
Fix the status code assertion
Globals Vault Tools
Buscar
ESP
LAA
89%
12:04
10/10/2026





Uso de IA

Inteligencia Artificial Utilizada: Gemini

1. Establecimiento de Rol

- **Prompt del usuario:** "Actúa como un programador de backend profesional"
- **Objetivo:** Definir el perfil y el contexto técnico de la conversación.
- **Información y palabras clave obtenidas:** Activación del rol de programador de backend, enfocado en arquitectura, diseño de APIs, bases de datos y microservicios.

- **Cómo llevó a la siguiente consulta:** Al fijar este perfil técnico, propiciaste plantear de inmediato un caso práctico de desarrollo en NestJS.

2. Diseño del Backend en NestJS y Archivo temporal.ts

- **Prompt del usuario:** "quiero hacer un backend en nest para realizar pruebas en POSTMAN (GET, POST, PUT, PATCH, DELETE, HEAD Y OPTIONS)... genere un nuevo recurso llamado usuarios, dentro del recurso usuarios crea un archivo llamado temporal.ts..."
- **Objetivo:** Construir la estructura inicial con un arreglo en memoria para manejar datos de usuarios y cubrir los 7 métodos HTTP solicitados.
- **Información y palabras clave obtenidas:** Estructura de NestJS, recurso usuarios, archivo temporal.ts con clase, constructor y arreglo inicial con id, nombre y edad, además de controladores y servicios asociados.
- **Cómo llevó a la siguiente consulta:** La entrega de este bloque de código motivó la necesidad de entender cómo ejecutar y verificar cada verbo HTTP en Postman.

3. Guía de Pruebas en Postman

- **Prompt del usuario:** "ahora quiero hacer las pruebas con todos los metodos de POSTMAN"
- **Objetivo:** Proveer instrucciones paso a paso sobre cómo configurar y ejecutar los 7 métodos HTTP en la herramienta.
- **Información y palabras clave obtenidas:** Endpoints, verbos (GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS), cuerpos en formato JSON y respuestas esperadas por el servidor.
- **Cómo llevó a la siguiente consulta:** Una vez comprendido el flujo de pruebas, el usuario solicitó una revisión exhaustiva del código fuente que sustentaba toda la lógica.

#	Tu Prompt o Consulta	Objetivo de la Consulta	Información / Palabras Clave Obtenidas	Cómo llevó a la siguiente consulta
1	"Actúa como un programador de backend profesional"	Establecer el rol técnico y la perspectiva del asistente.	Activación del rol de programador de backend (arquitectura, APIs, bases de datos).	Al fijar el perfil profesional, presentaste de inmediato un caso práctico de desarrollo de un backend en NestJS.

2	"quiero hacer un backend en nest para realizar pruebas en POSTMAN (GET, POST, PUT, PATCH, DELETE, HEAD Y OPTIONS). ahora quiero que escuches bien lo que te voy a decir: genere un nuevo recurso llamado usuarios, dentro del recurso usuarios crea un archivo llamado temporal.ts, en ese archivo esta una clase, constructor, array que tenga id, nombre, edad. y durante las pruebas de POSTMAN quiero que los datos (id, nombre, edad.) esten en el array para utilizar los metodos de POSTMAN"	Diseñar la estructura inicial en NestJS con un recurso usuarios, un archivo temporal.ts en memoria con una clase, constructor y datos de prueba para cubrir los 7 métodos HTTP.	Estructura en NestJS, recurso usuarios, archivo temporal.ts con la clase, constructor y arreglo de datos (id, nombre, edad), además de la configuración de controladores y servicios.	Al entregar el código y la estructura inicial, quisiste saber cómo ejecutar y probar todos esos métodos paso a paso en la herramienta de pruebas.

3	"ahora quiero hacer las pruebas con todos los metodos de POSTMAN"	Obtener una guía práctica para ejecutar y verificar cada uno de los 7 métodos HTTP en Postman.	Guía detallada con endpoints, verbos (GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS), cuerpos JSON y respuestas del servidor.	Conociendo el flujo de pruebas, solicitaste una revisión detallada instrucción por instrucción del código del archivo temporal.ts.
5	"Todos los MÉTODOS DE ACCESO Y MANIPULACIÓN DE DATOS tienes que ir obligado en el archivo?"	Resolver la duda de si la implementación de todos los métodos en el archivo temporal era un requisito estricto de NestJS o opcional.	Clarificación de que no son obligatorios y dependen estrictamente de los requerimientos de la API y las pruebas a realizar.	Al entender la flexibilidad de los métodos, preguntaste si podías moverlos a un archivo distinto.

6	"Puedo mover los MÉTODOS DE ACCESO Y MANIPULACIÓN DE DATOS en otro archivo?"	Averiguar si la lógica de datos podía trasladarse a otra ubicación siguiendo mejores prácticas.	Recomendación arquitectónica de mover la lógica directamente al Servicio de NestJS (usuarios.service.ts) para eliminar archivos auxiliares innecesarios.	Con base en los cambios discutidos, pediste un resumen de cuáles eran exactamente los archivos modificados.
7	"Dime qué archivos modificaste"	Identificar los archivos involucrados en la estructura que fuimos armando.	Listado de los 3 componentes lógicos del recurso: temporal.ts, usuarios.service.ts y usuarios.controller.ts.	Para visualizar de forma clara el contenido completo, solicitaste ver la descripción del código de esos 3 archivos juntos.

8	"De todos los 3 archivos que modificamos. en el mismo código describeme que está"	Obtener el código comentado de los tres archivos de la estructura de usuarios.	Bloques de código comentados explicando el propósito de temporal.ts, usuarios.service.ts y usuarios.controller.ts.	Finalmente, solicitaste estructurar un registro resumido de todo este proceso de investigación en forma de tabla con todas tus consultas.

FUENTES:

Componente / Aspecto Técnico	**Lo que implementó la IA / Tu Backend**	**Contraste con la Documentación Oficial**	**Enlace Oficial de Documentación**
1. Arquitectura de Controladores y Servicios	Estructuración en un controlador (usuarios.controller.ts) para enrutamiento, un servicio (usuarios.service.ts) para la lógica de negocio y una clase de persistencia temporal en memoria (temporal.ts) con un arreglo de datos.	**Separación de Intereses (SoC):** La documentación oficial de NestJS promueve el uso de Inyección de Dependencias. Los controladores manejan las solicitudes HTTP y delegan la lógica a los Providers (Servicios). El uso de arreglos en memoria en clases auxiliares es válido para fases de prototipado rápido antes de conectar una base de datos u ORM oficial.	Documentación Oficial de NestJS
2. Métodos HTTP (Verbos REST)	Implementación y verificación en Postman de los 7 métodos principales: GET, POST, PUT, PATCH, DELETE, HEAD y OPTIONS sobre el recurso usuarios.	**Semántica de Peticiones Web:** Según MDN, los métodos definen la acción sobre el recurso. GET y HEAD son seguros y no mutan el estado (HEAD solo retorna cabeceras). OPTIONS describe las opciones de comunicación (útil para CORS y cabecera Allow), mientras que POST, PUT, PATCH y DELETE gestionan el ciclo de vida del recurso.	MDN Web Docs - Métodos HTTP